

# 1 How Does Information Flow Secure Code?

1/17

## Information Flow in Programming

Track what data can influence,  
then reject flows that violate a security policy.

**Prevent bugs like:**

$\text{password}^H \rightsquigarrow \text{log}^L$      $\text{request}^H \rightsquigarrow \text{SQL}^L$      $\text{key}^H \rightsquigarrow \text{timing}^L$

**Core question:** if I observe  $y$ , what could have influenced it?  
dependency + policy = secure flow

## 2 Motivation: Confidentiality Leak

2/17

### CONFIDENTIALITY

Secrets should not influence public observations.

bad flow:  $\text{password}^H \rightsquigarrow \log^L \quad H \not\sqsubseteq L$

```
function login(req) {  
  const email = req.body.email;  
  const password = req.body.password;  
  
  console.log("login failed", { email, password });  
  return checkPassword(email, password);  
}
```

Violation: the log is observable, so logging password  
sends a secret to a public sink.

### 3 Motivation: Integrity / SQL Injection

3/17

#### INTEGRITY

Untrusted input should not control trusted operations.

bad flow:     req.query.name<sup>H</sup>  $\rightsquigarrow$  SQL interpreter<sup>L</sup>      $H \not\subseteq L$

```
function findUser(req) {  
  const name = req.query.name;  
  return db.query(  
    "SELECT * FROM users WHERE name = '" + name + "'"  
  );  
}
```

attacker input:     "x' OR '1'='1"     changes the query.

Violation: attacker data becomes database code.

## 4 Motivation: Timing Side Channel

4/17

bad flow:  $\text{secret}[i]^H \rightsquigarrow \text{runtime}^L \quad H \not\subseteq L$

```
function sameSecret(s, g) {  
  for (let i = 0; i < s.length; i++) {  
    if (s[i] !== g[i]) return false;  
  }  
  return true;  
}
```

The return time can reveal how many prefix bytes matched.  
This is implicit flow through control.

## 5 Definition: Influence and Dependency

5/17

$$x \rightsquigarrow e$$

means changing only  $x$  can change the value observed for  $e$ .

### SEMANTIC TWO-RUN TEST

hold every other variable fixed, vary only  $x$ ,  
then check whether the observed value of  $e$  changes

$$(\forall z \neq x. \sigma(z) = \sigma'(z)) \wedge \llbracket e \rrbracket(\sigma) \neq \llbracket e \rrbracket(\sigma') \Rightarrow x \rightsquigarrow e$$

$$\sigma \approx_L \sigma' \Rightarrow \llbracket P \rrbracket(\sigma) \approx_L \llbracket P \rrbracket(\sigma')$$

**Noninterference:** secret differences should not create public differences.

## 6 The Low/High Lattice

6/17

$L \sqsubseteq H$      $\ell_1 \sqsubseteq \ell_2$  means  $\ell_1$ -information may flow to  $\ell_2$

$$L \sqcup L = L \quad L \sqcup H = H \quad H \sqcup H = H$$

| Policy          | Low ( $L$ ) | High ( $H$ ) |
|-----------------|-------------|--------------|
| Confidentiality | Public      | Secret       |
| Integrity       | Trusted     | Untrusted    |

$H \not\sqsubseteq L$     Secret  $\not\rightsquigarrow$  Public  
Untrusted  $\not\rightsquigarrow$  Trusted

## 7 Explicit vs. Implicit Flow

7/17

### EXPLICIT FLOW

information moves through values

$y := x \quad x^H \rightsquigarrow y^H$

public = secret

### IMPLICIT FLOW

information moves through control

if secret then  $y := 1$  else  $y := 0$

observing  $y$  reveals *secret*

value labels track explicit flow

*pc* tracks implicit flow

## 8 Security-Typed STLC: Judgment Form

8/17

$$\Gamma; pc \vdash e : \tau^\ell$$

under environment  $\Gamma$  and control label  $pc$ , expression  $e$  has type  $\tau$  and label  $\ell$

$$\Gamma$$

$$x \mapsto \tau^\ell$$

Variable labels.

$$pc$$

control context

High branch means high context.

$$\tau^\ell$$

type plus flow label

A value carries a security level.

## 9 Core Typing Rules

9/17

### VARIABLE

$$\frac{\Gamma(x) = \tau^\ell}{\Gamma; pc \vdash x : \tau^\ell}$$

### CONSTANT

$$\overline{\Gamma; pc \vdash c : \tau^L}$$

### PRIMITIVE OPERATIONS

$$\frac{\Gamma; pc \vdash e_1 : \text{Int}^{\ell_1} \quad \Gamma; pc \vdash e_2 : \text{Int}^{\ell_2}}{\Gamma; pc \vdash e_1 + e_2 : \text{Int}^{\ell_1 \sqcup \ell_2}}$$

### BOOLEAN AND

$$\frac{\Gamma; pc \vdash b_1 : \text{Bool}^{\ell_1} \quad \Gamma; pc \vdash b_2 : \text{Bool}^{\ell_2}}{\Gamma; pc \vdash b_1 \wedge b_2 : \text{Bool}^{\ell_1 \sqcup \ell_2}}$$

### SUBSUMPTION

$$\frac{\Gamma; pc \vdash e : \tau^{\ell_1} \quad \ell_1 \sqsubseteq \ell_2}{\Gamma; pc \vdash e : \tau^{\ell_2}}$$

The result label is the join of everything that influenced it.

## CONDITIONALS

$$\frac{\Gamma; pc \vdash e : \text{Bool}^\ell \quad \Gamma; pc \sqcup \ell \vdash e_1 : \tau^{\ell_1} \quad \Gamma; pc \sqcup \ell \vdash e_2 : \tau^{\ell_2}}{\Gamma; pc \vdash \text{if } e \text{ then } e_1 \text{ else } e_2 : \tau^{pc \sqcup \ell \sqcup \ell_1 \sqcup \ell_2}}$$

$$pc' = pc \sqcup \ell$$

## WRITING TO AN OUTPUT SINK

$$\frac{\Gamma; pc \vdash e : \tau^\ell \quad pc \sqcup \ell \sqsubseteq \ell_s}{\Gamma; pc \vdash \text{write}_{\ell_s}(e) : \text{Unit}^L}$$

$\ell_s$  labels where data becomes observable: log, file, response, query

$\ell$  tracks explicit flow       $pc$  tracks implicit flow

$pc \sqcup \ell \sqsubseteq \ell_s$  means every influence is allowed at that sink

## 11 Example 1: Secret Condition

11/17

### EXPRESSION

$$e_1 = \text{if secretFlag}^H \text{ then } 0^L \text{ else } 1^L$$

### STEP BY STEP

$$\begin{aligned} pc &= L & \ell_{\text{cond}} &= H & \ell_{\text{then}} &= L & \ell_{\text{else}} &= L \\ \text{label}(e_1) &= pc \sqcup \ell_{\text{cond}} \sqcup \ell_{\text{then}} \sqcup \ell_{\text{else}} = L \sqcup H \sqcup L \sqcup L = H \end{aligned}$$

**Result level:**  $H$

The branch values are public, but the chosen branch reveals information about the secret condition.

## 12 Example 2: Secret Branch

12/17

### EXPRESSION

$$e_2 = \text{if } \text{publicFlag}^L \text{ then } 0^L \text{ else } \text{secretValue}^H$$

### STEP BY STEP

$$\begin{aligned} pc &= L & \ell_{\text{cond}} &= L & \ell_{\text{then}} &= L & \ell_{\text{else}} &= H \\ \text{label}(e_2) &= pc \sqcup \ell_{\text{cond}} \sqcup \ell_{\text{then}} \sqcup \ell_{\text{else}} = L \sqcup L \sqcup L \sqcup H = H \end{aligned}$$

**Result level:**  $H$

The condition is public, but one possible returned value is secret, so the whole expression is high.

## 13 Example 3: Mixed Addition

13/17

### EXPRESSION

$$e_3 = \text{public}A^L + \text{private}B^H + \text{public}C^L$$

### STEP BY STEP

$$\begin{aligned} \ell_A &= L & \ell_B &= H & \ell_C &= L \\ \text{label}(e_3) &= \ell_A \sqcup \ell_B \sqcup \ell_C = L \sqcup H \sqcup L = H \end{aligned}$$

**Result level:**  $H$

Two public terms do not hide one private influence;  
the whole arithmetic result must be treated as high.

## 14 From Type Rules to Data-Flow Analysis

14/17

$$\begin{aligned} CFG &= (N, E) & \text{In}[n], \text{Out}[n] : \text{Var} \rightarrow \{L, H\} \\ \text{In}[n] &= \bigsqcup_{p \in \text{pred}(n)} \text{Out}[p] & \text{pc}[n] = \bigsqcup_{g \in \text{Guards}(n)} \text{label}(g) \end{aligned}$$

### TRANSFER (GEN/KILL)

$$x := e \quad \text{label}(e) = \bigsqcup_{y \in FV(e)} \text{In}[n](y)$$

$$\text{Gen}[n] = \{x \mapsto \text{pc}[n] \sqcup \text{label}(e)\}$$

$$\text{Kill}[n] = \{x\} \text{ (strong), } \emptyset \text{ (weak)}$$

$$\text{Out}[n] = (\text{In}[n] \setminus \text{Kill}[n]) \sqcup \text{Gen}[n]$$

### SECURITY POLICY

$$\text{source}() : H$$

$$\text{sink}_L(e)$$

$$\text{sanitize}(e) : L$$

$$\text{pc}[n] \sqcup \text{label}(e) \not\sqsubseteq L \Rightarrow \text{report flow}$$

$\text{pc}[n]$  join guard labels controlling whether  $n$  runs  
 $\text{Gen}[n]$  generate new influences from  $e$  and  $\text{pc}[n]$   
 $\text{Kill}[n]$  kill old influences only for a definite overwrite

## 15 Alias Analysis: Following the Heap

15/17

$$\text{Pt}(x) = \{o_1, o_2, \dots\}$$

Alias analysis tells information flow which names may touch the same storage.

### FIELD WRITE

$$x.f := e$$

$$o \in \text{Pt}(x)$$

$$\text{label}(o.f) := \text{label}(o.f) \sqcup pc \sqcup \text{label}(e)$$

### FIELD READ

$$y := x.f$$

$$\text{label}(y) = \text{label}(x) \sqcup \bigsqcup_{o \in \text{Pt}(x)} \text{label}(o.f)$$

$$\text{Pt}(a) \cap \text{Pt}(b) \neq \emptyset$$

$\Rightarrow$   $a.f$  and  $b.f$  may be the same channel.

## 16 Applications and Current Limits

16/17

### CONFIDENTIALITY

Secret  $\nleftrightarrow$  Public

password  $\nleftrightarrow$  logs

### INTEGRITY / TAINT

Untrusted  $\nleftrightarrow$  Trusted

request  $\nleftrightarrow$  SQL

### SIDE CHANNELS

Secret  $\nleftrightarrow$  behavior

key bit  $\nleftrightarrow$  timing

Hard parts: policy specification, alias precision, implicit flows, side channels, libraries/frameworks, false positives/negatives, and scalability.

**Takeaway:** information flow = dependency + policy + analysis

## 17 Research Extension: Actor-Aware Policies

17/17

Low/high labels say which flows are allowed;  
actor-aware policies also ask who has permission.

### ACTORS

$a \in \text{Actor}$      $op \in \{\text{read}, \text{write}\}$

Who is using the sink?

### PERMISSION CONSTRAINTS

$\text{perm}(a, op, s)$

Is actor  $a$  authorized for  $s$ ?

### EXTENDED SINK CHECK

$a \vdash \text{write}_s(e) \text{ ok iff } pc \sqcup \text{label}(e) \sqsubseteq \ell_s \quad \wedge \quad \text{perm}(a, \text{write}, s)$

**Research direction:** flow policy = label constraints + actor permissions